

CitGraph: Citation Intent Classification and Second-Hand Citation Detection via Knowledge Graphs

Ethics in Artificial Intelligence
University of Bologna

Giorgio Scavello

`giorgio.scavello@studio.unibo.it`

0001169261

Lorenzo Pellegrino

`lorenzo.pellegrino2@studio.unibo.it`

0001189550

August 26, 2026

Abstract

Citations are usually treated as an undifferentiated signal, even though their role depends on why and how they were made. This project introduces CitGraph, a system that reframes citations as edges in a graph rather than as raw counts, combining graph structure with citing sentence text to characterise and audit them. Starting from SciCite, the raw splits are expanded via the Semantic Scholar API, enriched with node features, and corrected for temporally inconsistent edges. An MLP baseline, GraphSAGE, and SEAL are then trained to predict link existence under hard negatives, with GraphSAGE powering an explainable web application that ranks and justifies candidate citations. A second task flags likely second hand citations by comparing citing sentence similarity within closed triads against a per target null model, browsable through a companion application. The report closes by reading these design choices against explicability, robustness, fairness, and accountability.

1 Introduction

Scientific literature grows faster than any researcher can read, and citations are the main way this growing body of work gets connected and filtered. Yet a citation is usually treated as an undifferentiated signal. Counting citations, ranking papers by them, or building recommenders around them all assume that every citation carries the same weight, when in practice a citation’s role depends heavily on why and how it was made.

This has two consequences. Interpretively, a paper that shaped a subfield’s methodology and one cited only in passing as background can end up looking identical once reduced to a single count, so raw citation counts say little about actual influence. From a verification standpoint, as bibliographies grow under pressure to appear thorough, a citation increasingly reflects familiarity with how another paper already frames a source rather than direct engagement with it, and such secondhand references can travel through the literature looking no different from ones grounded in independent reading. Both problems trace back to the same cause. A citation read in isolation, stripped of its surrounding sentence and of the other papers that also cite the same work, carries almost none of the context needed to judge its role or its genuineness.

This report addresses that problem by treating the literature as a graph of papers connected through citations, using the structure of that graph together with the text around each citation to characterise and audit citations individually. CitGraph, the resulting system, applies this idea to a public citation dataset and packages it into predictive models and two interactive tools, detailed in the sections that follow.

2 Related work

Prior work falls into two strands relevant to CitGraph’s two contributions. To our knowledge, no existing research addresses the problems listed here in their entirety.

2.1 Citation-based literature recommendation

SciCite [3] established the three-class scheme this project inherits, using only the citing sentence as input. Its goal was to predict citation intent from text alone; it does not construct any graph over papers, so intent is inferred without any relational context between citations.

GraphCite [2] is the closest prior attempt to bring graph structure into this task. Its best-performing model, CitationBERTGAT, combines two largely independent branches: a BERT-based encoding of the citing sentence, and node embeddings produced by a Graph Attention Network (GAT) over the citation graph, concatenated only at a later stage. Unlike CitGraph, where the citing-sentence embedding is itself an edge feature the GNN aggregates over during message passing, GraphCite’s text and graph signals are fused late rather than integrated into the same propagation step.

In recent years, tools such as ResearchRabbit and Connected Papers expand outward from a seed paper along citation links but rank suggestions opaquely. PUREsuggest [1] names this shortcoming explicitly and responds with a transparent, hand-designed ranking score. This closes the gap for the heuristic itself, but not for a trained predictor: it cannot explain why a specific link-existence model scored a specific candidate pair, and is therefore lacking in explainability. Nor do these tools counteract the consequence of ranking papers by a single score, usually based on citation count, suggesting always the highly-cited, well-established papers leaves newer or less-cited but still relevant work buried further down the list.

2.2 Detecting unread, copied citations

Simkin and Roychowdhury [7] showed that repeated misprints across citing papers imply that 70 to 90% of citations are copied from another paper’s reference list rather than read firsthand. This detects copying only indirectly, through transcription errors, and never identifies the intermediate paper a citation was copied through.

3 Technologies

The whole project is implemented with Python files/modules, and the processes are documented with Jupyter Notebooks. In particular, some phases required specific Python modules:

- **igraph**: primary data structure for the representation of citation graphs. It is used instead of the classic **NetworkX** because it is a C-backed library, which is considerably lighter on memory for our node counts and supports directed multigraphs (parallel edges) natively, together with fast weakly/strongly-connected-component queries.
- **requests**: we needed it to query the **Semantic Scholar Graph API**¹[6] to perform the one-hop graph expansion (fetching references and citations of seed papers) and to fetch per-paper metadata and embeddings.
- **pickle** and **SQLite3**: we needed them to store intermediate results, allowing the long-running expansion and enrichment process to be check-pointed and resumed. The former was used to load entire graphs or small intermediate API caches, while the latter was used for the enrichment procedure, which was too heavy to be loaded entirely in RAM.

¹<https://api.semanticscholar.org/graph/v1>

4 Methodology

General description of the system architecture and methodology.

4.1 Project Folder Structure

The project, as illustrated in 1, has this structure:

1. **scicite**: contains the original train, validation and test graphs splits as illustrated in the original paper.
2. **graphs**: contains the creation of the graphs splits actually used in the project.
 - **graphs_generation.ipynb**: explain the pipeline that modifies the original graphs and generates the new ones after cleaning, disjunction, expansion, enrichment, correction and intermediate operations.
 - **cache**: the graphs are not that small and the modifications only increase their sizes. In order to help the pipeline development procedure, we needed many checkpoints and the final one (**final_graphs.pkl**) will be directly used by the neural networks.
3. **scripts**:
 - **common/text_embedding.py**: fetch title/abstract per paper and turn it into SciBERT embeddings, shared by the second-hand citation task and by the graph-generation notebook.
 - **hard_negatives.py**: shared generation of structurally-plausible ("hard") negative pairs, used by the **_hard** training scripts and by **eval_hard_negatives.py**.
 - **train_mlp_baseline_{raw,encoded}.py**: train the two MLP baseline variants (Section 4.4) with random negatives.
 - **train_graphsage.py** / **train_seal.py**: train a link-existence predictor with random negatives, using GraphSAGE and SEAL respectively.
 - **train_mlp_baseline_{raw,encoded}[_hard].py**: retrain the same four models with hard negatives and a ranking loss instead of random negatives.
 - **eval_hard_negatives.py**: re-evaluates the random-negative checkpoints above against hard negatives.
 - **tune_hyperparams.py**: grid-search driver for the four training scripts.
 - **second_hand_citation.py**: downstream analysis task, detecting candidate second-hand citations via citing-sentence similarity.
4. **GNNs**: contains the operations with the Graph Neural Networks,
 - **train_and_test.ipynb**: invokes the training functions from the scripts using the generated graphs.
 - **weights**: contains the GNNs parameters after the training phase, in **.pt** format.
 - **results**: contains the GNNs results after the training/validation and test phases, in **.csv** format.
5. **link_predictor_webapp**: the standalone web application described in Section 4.4 that serves the link-existence predictor.
 - **app.py**: the lightweight serving process, re-implementing the decoder's forward pass over precomputed embeddings.

- `build_recommender_index.py`: the offline precomputation script that embeds every paper once and exports the embeddings and decoder weights the app reads at request time.
6. `second_hand_citation_webapp`: the companion web application described in Section 4.5 that makes flagged triads browsable.
- `app.py`: the lightweight serving process, reading the precomputed table of scored triads.
 - `build_graph_index.py`: builds the citation-graph index used to render each triad’s surrounding context.

4.2 Original datasets and their inspection

The project builds on SciCite [3], a dataset of citation contexts extracted from computer science and biomedical papers taken from Scholar, in which each citation is annotated with the rhetorical role it plays in the citing sentence. SciCite is distributed as three JSON files, one per split (`train`, `dev`, `test`), stored under `scicite/`. Each line is a single citation instance with the following fields:

- `citingPaperId` and `citedPaperId`: Semantic Scholar paper identifiers for the citing and cited paper, respectively the two endpoints of a citation edge.
- `string`: the raw text of the citation context (the sentence, or span, containing the citation).
- `label`: the primary citation-intent label, the values are:
 - `background`: the citation states, mentions, or points to background information giving more context about a problem, concept, approach, topic, or the importance of the problem in the field.
 - `method`: the citing paper makes use of a method, tool, approach, or dataset from the cited paper.
 - `result`: the citing paper compares its results or findings with the results or findings of the cited paper.
- `label_confidence`: an annotator-confidence score for `label`, present for most but not all rows.
- `label2` and `label2_confidence`: a secondary annotation layer (*supportive*, *not_supportive*, *cant_determine*), present only for a small subset of rows.
- `citeStart`, `citeEnd`, and `excerpt_index`: character offsets locating the citation within the excerpt, and its position among multiple citations sharing the same excerpt.
- `sectionName`: the paper section the citation appears in (e.g. Introduction, Methods).
- `source`: how the citation was referenced in the text: *explicit* (a citation marker), *properNoun*, *acronym*, *acronymParen*, *andPhrase*, or *etAlPhrase*.
- `isKeyCitation`: a boolean flag for whether the citation was marked as a key reference, it is computed by Scholar.
- `id`: a unique identifier for the individual annotated instance (the specific citation), distinct from the paper-level `citingPaperId/citedPaperId`.

```

pellegrinoscavello2425
├── scicite
│   ├── train.jsonl
│   ├── dev.jsonl
│   ├── test.jsonl
│   └── scaffolds
│       ├── cite-worthiness-scaffold-train.jsonl
│       └── sections-scaffold-train.jsonl
├── graphs
│   ├── graphs_generation.ipynb
│   ├── cache
│   │   └── final_graphs.pkl
│   └── plots
├── scripts
│   ├── data_utils.py
│   ├── hard_negatives.py
│   ├── eval_hard_negatives.py
│   ├── tune_hyperparams.py
│   ├── second_hand_citation.py
│   ├── train_mlp_baseline_{raw,encoded}[_hard].py
│   ├── train_graphsage[_hard].py
│   ├── train_seal[_hard].py
│   └── common
│       ├── training_loop.py
│       ├── hard_negative_eval.py
│       ├── text_embedding.py
│       └── atomic_cache.py
├── GNNs
│   ├── train_and_test.ipynb
│   ├── weights
│   └── results
├── link_predictor_webapp
│   ├── app.py
│   ├── build_recommender_index.py
│   ├── templates
│   └── static
└── second_hand_citation_webapp
    ├── app.py
    ├── build_graph_index.py
    ├── templates
    └── static

```

Figure 1: The essential project folder structure.

After inspecting the splits we summarized their composition in Table 1. **train** contains the vast majority of unique papers, while **test** and **dev** are considerably smaller and comparable in scale to each other. The primary **label** distribution is consistent across splits and heavily skewed toward *background* citations, followed by *method* and *result*. The **source** field is similarly dominated by *explicit* citations, with the remaining categories each accounting for a small minority.

Table 1: Size and composition of the raw SciCite splits.

Split	Instances	Citing papers	Cited papers	Unique papers	label			source	
					background	method	result	explicit	other
Train	8,243	4,664	4,557	9,221	4,840	2,294	1,109	7,886	355
Test	1,861	1,845	1,804	3,649	997	605	259	1,659	199
Dev	916	817	802	1,619	538	255	123	882	33

Two properties of the raw data are worth calling out explicitly, since they directly motivate the first two stages of the graph-generation pipeline (Section 4.3):

- **Some attributes are not usable:** both `label2` and `label2_confidence` are populated for only a small fraction of rows, so this secondary annotation layer is too sparse to use as a training signal. `label_confidence` is likewise missing for a non-trivial portion of rows, so even this is not usable either.
- **Some citations/references are not usable:** some rows have a `None` value for `citedPaperId` or `citingPaperId`, in the same proportion across all three splits.

4.3 Graph generation

SciCite’s citation instances are a flat list of citation contexts, not a graph: turning them into graph structures suitable for GNN training required a multi-stage pipeline, implemented in `graphs_generation.ipynb`:

1. inspection and cleaning of the raw splits.
2. construction of fully disjoint graphs to remove leakage from train to test/dev.
3. topology-aware enrichment via the Semantic Scholar API to counteract the extreme sparsity of the raw data.
4. node-feature attachment.
5. final correction pass for temporally-inconsistent edges.

Each stage is detailed below.

4.3.1 Inspecting and cleaning the raw splits

As we anticipated in the previous section, rows with a missing `citedPaperId`/`citingPaperId` were dropped first, as they carry no usable edge. More importantly, the three splits are not disjoint at the paper level: computing the union of citing/cited identifiers per split and intersecting them pairwise showed 936 papers shared with the training set alone. Since a paper appearing in both `train` and `test` would let a GNN implicitly memorize test-set structure during message passing, this overlap constitutes a direct information leakage that later stages are specifically designed to remove.

4.3.2 Enforcing disjoint splits

To eliminate the leakage above, every contested paper (one occurring in more than one split) is assigned to a single *owning* split, and any citation edge that would cross the resulting split boundary is dropped. The assignment is randomized with a fixed seed for reproducibility, but the sampling weight for each split is inversely proportional to that split’s size. This is a deliberate choice: under uniform random assignment, a small split such as `dev` would systematically lose a

disproportionate share of its contested papers to the much larger **train** split, further shrinking it. Weighting inversely by size compensates for this and keeps the split sizes closer to their original proportions. Applying this procedure yields three pairwise-disjoint graphs with a loss of roughly 15% of train’s original papers, 8% of test’s, and nearly 20% of dev’s, confirming that even with inverse-size weighting the smallest split still pays the highest relative price for disjointness.

4.3.3 The sparsity problem and one-hop expansion

Since node count and edge count are roughly equal in all three disjoint graphs, the raw graphs are extremely sparse: over 99% of nodes sit outside any giant connected component. This is a direct consequence of SciCite only recording citation *contexts* that were manually annotated, not a paper’s full reference list, most nodes are therefore only connected to a single paper that cites or is cited by them.

A graph this disconnected blocks a message-passing GNN from propagating any signal beyond isolated pairs, so each split is expanded by one hop using the Semantic Scholar Graph API: for every seed node, both its outgoing references² and incoming citations³ are fetched and added as new nodes and edges.

Each new edge carries two attributes returned directly by the API:

- **contextsWithIntent**, the list of textual contexts in which the citation occurs together with an automatically-inferred intent label (analogous to SciCite’s own **label**)
- **isInfluential**, a boolean flag mirroring SciCite’s **isKeyCitation** that marks whether the citing paper is considered to substantively build on the cited one rather than merely mention it

The three splits are expanded independently and never share seed nodes, but this alone does not guarantee the expanded graphs remain disjoint, since a highly-cited paper can easily be pulled in as a shared neighbor of seeds from two different splits. Disjointness at this stage is therefore only re-established afterwards, with the identical size-weighted assignment rule described before, now applied to the expanded node sets. A request-level cache (keyed by paper id, shared only to avoid redundant API calls) and a custom **RateLimiter** enforce Semantic Scholar’s authenticated rate limits and make the whole fetch resumable across notebook runs.

The resulting graphs grow substantially, following Table 2: train gains $\sim 82\times$ nodes and $\sim 158\times$ edges, test $\sim 83\times$ and $\sim 242\times$, dev $\sim 80\times$ and $\sim 216\times$. Moreover the giant-components ratios jump from 0.1–0.2% to 76–90% for train and test. However, **dev** lags well behind at only 21.8%: of the $\sim 103k$ nodes it gained through expansion, just 21.8% landed inside the giant component, the rest scattered outside it as new leaves attached to seeds that never merged with anything else.

Since the giant component that did form is almost entirely new nodes rather than original seeds, a second, targeted expansion round is applied to **dev** only: well-connected, not-yet-queried nodes already inside its giant component are expanded by a further hop, with disjointness enforced inline against **train** and **test** during discovery rather than after the fact.

Isolated (degree-0) nodes are then dropped from all three splits, as they carry no structural signal. This brings **dev**’s giant-component ratio up to 64%, much closer to **train** (91%) and **test** (77%), while leaving the other splits’ statistics essentially unchanged.

²GET /paper/{id}/references

³GET /paper/{id}/citations.

Table 2: Graph statistics across the three expansion phases: pre-expansion, post one-hop expansion, and final.

Phase	Split	Nodes	Edges	Giant-comp. ratio
Pre-expansion	train	7,845	6,016	0.1%
	test	3,337	1,678	0.1%
	dev	1,299	715	0.2%
Post 1-hop expansion	train	639,680	953,437	89.9%
	test	275,438	405,888	76.0%
	dev	104,590	154,703	21.8%
Final (dev 2 nd -hop + cleanup)	train	629,783	953,437	91.3%
	test	272,024	405,888	76.9%
	dev	132,770	218,928	64.2%

A closer look⁴ at non-giant components also reveals a clear hub-and-spoke morphology rather than long citation chains: the fraction of degree-2 (pass-through) nodes is low across all component sizes (Fig. 2), linearity (diameter over size) decays roughly as $1/\text{size}$ (Fig. 3), and for components small enough to compute exactly, the median distance from each component’s highest-degree node to every other node is 2 hops, with the large majority of nodes reachable in a single hop from that hub. This favors a GNN’s ability to propagate information, since most nodes sit close to a well-connected hub rather than at the end of a long, information-diluting chain.

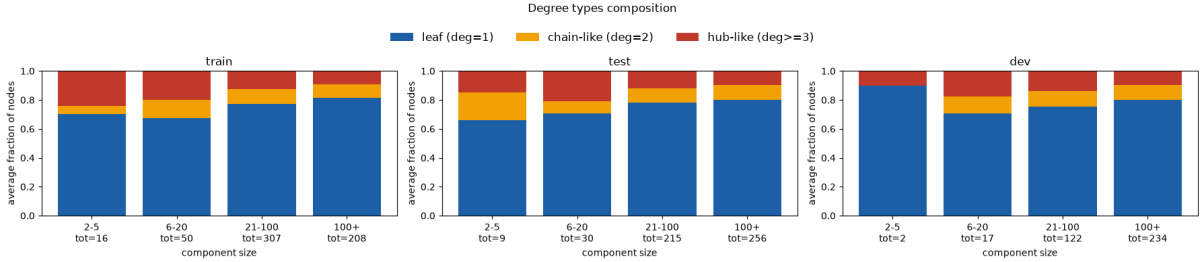


Figure 2: Components degrees per split, divided by number of nodes per component

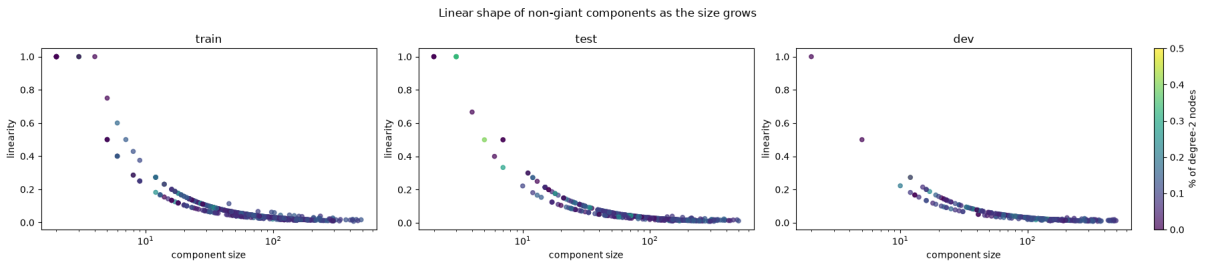


Figure 3: Diameter-to-size ratio (linearity) as a function of component size, per split.

4.3.4 Node feature enrichment

Up to this point, the expansion step only added topology (nodes and citation edges), every node still carried nothing particularly useful for the neural networks. Per-paper attributes are then fetched in bulk from the Semantic Scholar batch endpoint⁵ and attached as node features:

⁴The huge number of nodes makes plotting impracticable on final graphs, so we use statistics.

⁵POST /paper/batch

- `embedding.specter_v2`: a 768-dimensional, citation-aware SPECTER2 [4] embedding computed from title and abstract, this is the node-level counterpart to the sentence-level embeddings already used as edge features.
- `year` and `publicationDate`: `year` supports temporal ordering, while `publicationDate` resolves same-year ties at no extra query cost and is later needed to correct temporally-inconsistent edges (Section 4.3.5).
- `venue` and `s2FieldsOfStudy (domain)`: bibliometric context features.
- `citationCount` and `referenceCount`: free in the same call, and a standard proxy for a paper’s centrality/importance.
- `authors` (author ids only): stored as raw material for a possible future co-authorship signal and to allow copyright protection.
- `title`: kept for producing human-readable explanations rather than as a numeric feature.

Because materializing every attribute for over a million nodes in memory was infeasible (an estimated 5 GB), fetched records are written incrementally to an on-disk SQLite store instead, with the embedding vector already compacted to `float32` before being pickled. The downstream code reads one paper’s record at a time from this store rather than holding the full dataset in RAM, which also makes the fetch safely resumable after an interruption.

A post-enrichment audit checked both coverage and plausibility of the fetched attributes. Domain distribution is closely matched across splits, which is reassuring for generalization from train to dev/test. The one attribute with a non-negligible gap is the SPECTER2 embedding, missing for 5.5–6.0% of nodes in every split. Since embeddings are the primary node feature for the GNNs, these nodes were characterized before deciding how to handle them: they skew low-degree and, critically, none of them are articulation points inside their split’s giant component. A direct simulation of dropping them confirmed the giant-component ratio is essentially unchanged after removal (91.3% before vs. 90.6% after in train), so all nodes with a missing embedding were dropped, together with any neighbor left at degree zero as a result. This produced a combined loss of 36,200 / 14,959 / 8,014 nodes in train/test/dev, with no measurable structural cost.

4.3.5 Correcting temporally-impossible edges

A directed citation graph should be acyclic with respect to publication time, since a paper cannot cite one published after itself. The presence of any non-trivial strongly-connected component (SCC) is therefore, by construction, a sign of erroneous data rather than a real phenomenon. Every non-trivial SCC in each split (now smaller, since this pass runs on the post-enrichment graphs after the embedding-driven pruning above) was decomposed into its simple cycles, and each cycle was checked for a paradoxical edge (one going from an older to a strictly younger paper).

The time gap responsible for these paradoxes is rarely a borderline same-period case: the worst hop in each cycle falls in the 2+ years bucket for 65.3% of train’s paradoxical cycles and 67.6% of test’s, ruling out simultaneous cross-citation as the explanation. `dev` is somewhat milder at 47.8%, though still far from a near-boundary artifact. Almost all of these paradoxical cycles are simple reciprocal pairs (both $A \rightarrow B$ and $B \rightarrow A$ edges): 100% in train and test, and 91.3% in `dev` (the remaining 8.7% being length-3 cycles).

For each reciprocal pair, the edge pointing from the older to the younger paper is the physically impossible direction and is removed, leaving the valid (younger-citing-older) edge in place. Cycles longer than 2 nodes are left untouched, since they are very few and with more than two nodes involved there is no single edge that can be unambiguously blamed, and a case-by-case treatment

was judged out of scope for this stage. This removed only a negligible fraction of each split’s total edge count, yet it eliminates a topological inconsistency that a directed GNN would otherwise have to reconcile on its own.

4.4 Link-existence prediction

The graphs produced by the pipeline in Section 4.3 only tell us the state of the art, i.e., the citations already performed across the different papers. However, the main interest here is to suggest papers that could be of interest for the researcher’s current work, based on the paper(s) they have already cited. For this reason, using just the graph is informative but not sufficient: it would be more useful to also surface related research that was not cited by the paper being read, yet is still relevant. More importantly, the system should be able to suggest research that the original paper could not have cited because it was published later in time; this would allow the user to stay up to date on the latest developments in the field in an easy and accessible way. To achieve this goal, three different models were used: an MLP baseline, which does not exploit the graph structure, a GraphSAGE encoder, and SEAL, another encoder which, unlike GraphSAGE, scores a candidate pair of nodes by extracting their enclosing subgraph. The webapp deploys then GraphSAGE as its employment does not require online computation.

As mentioned in Section 2, state-of-the-art tools such as Research Rabbit do not explain their suggestions, leaving the user with no way to judge how confident to be in a suggested paper or why it was recommended in the first place. Building on this observation, in the webapp a Symbolic Knowledge Extraction (SKE) decision tree was fit. This is used to explain the main features driving each suggestion, falling back to a per-instance LIME explanation whenever the tree’s prediction disagrees with the real model’s output for that specific case, or when the tree is not reliable enough overall to be trusted.

After a brief description of the training setup, each of the methods introduced above is explained in detail in this section, including the previously mentioned webapp.

4.4.1 Task definition and shared node features

All three models, the MLP baseline, GraphSAGE, and SEAL, are trained and evaluated following the standard pattern: the model is fit on **train**; model selection (checkpointing and early stopping) is driven by performance on the whole **dev** graph (papers the encoder never trains on), evaluated after every epoch; and **test** is scored exactly once, at the end, untouched by any selection decision. Every node carries two kinds of features, concatenated into a single vector: a content feature, the 768-dimensional SPECTER2 embedding already attached during enrichment, and a small set of structural features, namely log-scaled in-degree, log-scaled out-degree, and a binary flag marking whether the node was part of the original SciCite seed set or was only added by the expansion step. The SPECTER2 embedding is a legitimate per-node feature for this task specifically because it is computed from a paper’s own title and abstract: it exists independently of any citation edge, so using it does not leak “this edge exists” information into a negative (non-existent) pair the way an edge-level feature would.

4.4.2 The random-negative problem and hard negatives

All three models above are trained with negatives sampled uniformly at random from all non-edges. A uniformly-random non-edge (u, v) is usually an easy negative: u and v are typically unrelated in topic and far apart structurally, so a model with access to both content and structure can tell it apart from a real citation with little effort. This makes random-negative performance a weak proxy for how a model would do on a genuinely ambiguous candidate pair, which is precisely the regime the downstream second-hand-citation task operates in (Section 4.5), since a second-hand citation is by construction a plausible, structurally-close pair. A hard negative

is defined here as an open 2-hop triad: given an intermediate paper w , pick u such that $u \rightarrow w$ is a real edge and v such that $w \rightarrow v$ is a real edge, and keep the pair (u, v) only if $u \rightarrow v$ is not itself a real edge. Such a pair is structurally plausible (u is one citation-hop away from v through w) without being a real citation, making it a much closer stand-in for the kind of pair the models eventually need to distinguish. Candidate generation caps how many pairs a single intermediate paper w can contribute, since a small number of highly-cited hub papers would otherwise dominate the sample and inflate runtime disproportionately. To make use of this harder negative type during training itself, rather than only at evaluation time, all three models, MLP baseline, GraphSAGE, and SEAL, are retrained with an alternative objective. For every real training edge (u, v) , one matched hard negative (u, v') sharing the same anchor u is sampled from u 's 2-hop-reachable-but-not-cited targets (falling back to a uniformly-random target when u has none), and the model is trained with a pairwise Bayesian Personalized Ranking (BPR) loss

$$-\log \sigma(\text{score}(u, v) - \text{score}(u, v'))$$

which is implemented as

$$\text{softplus}(\text{score}(u, v') - \text{score}(u, v))$$

for numerical stability. Unlike binary cross-entropy, which pushes every pair's score independently toward a fixed 0/1 threshold, a pairwise ranking loss directly optimizes $\text{score}(u, v) > \text{score}(u, v')$ for the same anchor u , matching what "hard negative" means here: the model must learn to distinguish "almost cited, through an intermediate paper" from "actually cited", not merely separate two unrelated classes.

4.4.3 Evaluation metrics

Every model is evaluated with two metrics, both computed from the continuous predicted probabilities $\text{sigmoid}(\text{score}(u, v))$ rather than from a thresholded 0/1 decision: **ROC-AUC** and **average precision (AP)**, via `scikit-learn`'s `roc_auc_score` and `average_precision_score`. No accuracy, F1, or $\text{precision}@k$ is reported, since the goal is to rank candidate pairs by plausibility rather than to make a hard cite/don't-cite call for any single pair. For the random-negative-trained models, dev and test evaluation sets are built the same way as training, at a 1:1 positive-to-negative ratio (`RandomLinkSplit`'s default `neg_sampling_ratio=1.0`). For the hard-negative-trained models, dev and test are instead built from matched 2-hop hard negatives, one per positive anchor edge (falling back to a random negative when an anchor has no 2-hop candidate), again keeping a 1:1 ratio so AP stays comparable across settings. A separate, non-retraining script re-scores the already-trained random-negative checkpoints against the same hard-negative evaluation sets, explicitly to test how badly a model trained only on easy negatives degrades when faced with structurally-plausible ones: this comparison is what actually substantiates the random-negative weakness argued for in Section 4.4.2, rather than assuming it. Model selection during training is driven by dev-set AP rather than dev-set AUC, consistent with the `es_ap` suffix on every released checkpoint: after every epoch, dev AUC and AP are both computed, but the early-stopping criterion and the checkpoint that gets saved are chosen by AP specifically, since AP is more sensitive to performance on the minority class of true positive edges relative to non-edges, which matters more here than an aggregate ranking measure that treats all pairs equally. The test split is touched exactly once, after training has finished and the best dev-AP checkpoint has been reloaded, so the reported test numbers are never used to make any decision during training.

4.4.4 Hyperparameter tuning

Each of the eight training configurations (MLP baseline raw, MLP baseline encoded, GraphSAGE, and SEAL, each in both its random-negative and hard-negative variant) is tuned via

a grid search over a small, model-specific set of hyperparameters: learning rate, hidden dimension, number of layers, and either weight decay (MLP baseline, GraphSAGE) or DRNL label-embedding dimension (SEAL). A grid search rather than a random search was chosen deliberately, since each parameter takes only two or three discrete values, keeping the full grid (up to a few dozen combinations per configuration) exhaustively enumerable. Every combination is run as an independent training trial, model-selected on `dev` under the same early-stopping criterion used for the final fit. The tuning trials are literally invocations of the same training script, so tuning and the final training run share identical code, not just a similar setup. Only the best-performing trial’s checkpoint is kept, identified by its `dev` average precision (AP), consistent with the metric-selection choice described in Section 4.4.3.

4.4.5 MLP baseline

The first model is an MLP baseline that deliberately discards the graph structure, serving as a floor against which the graph-based encoders (GraphSAGE, SEAL) can be measured: any improvement they show can then be attributed to actually exploiting citation structure, rather than to the node features or the pairwise decoder alone. Two variants are trained. The raw variant feeds each node’s own content+structural feature vector directly into the pairwise decoder, with no per-node transformation at all: $h_v = x_v$. The encoded variant first passes x_v through a small `NodeMLPEncoder`, a stack of `Linear` layers of the same depth and width as the GraphSAGE encoder (2 layers, 128-dimensional hidden state by default), with ReLU and dropout applied after every layer except the last.

Crucially, this encoder operates on each node strictly independently: it takes only x_v as input, never the edges, so no information can cross between nodes. `edge_index` is accepted as a parameter in the model’s `forward` call purely to keep the interface identical to the GraphSAGE model’s, but it is never read. Comparing the two variants isolates whether a learned nonlinear per-node representation helps at all ("encoded" vs. "raw"), independently of whether neighborhood aggregation helps (GraphSAGE/SEAL vs. either MLP variant). Both variants score a candidate pair (u, v) with the same decoder architecture used by GraphSAGE: the concatenation $[h_u || h_v]$ passed through a small MLP (`Linear` $\rightarrow$ ReLU $\rightarrow$ Dropout $\rightarrow$ `Linear`), with sigmoid applied externally to produce a probability. Training follows the identical protocol described in Section 4.4.2 and the task definition above: negatives are sampled as directed non-edges (`RandomLinkSplit(..., is_undirected=False)`), the model is fit on `train`, model selection uses `dev` performance evaluated after every epoch, and `test` is scored once, at the end. The only difference from GraphSAGE’s training setup is the encoder itself: a per-node MLP (or the identity function, for "raw") in place of a message-passing `SAGEConv` stack.

4.4.6 GraphSAGE

The second model is a GraphSAGE encoder [5], which learns to produce node embeddings by repeatedly aggregating information from a node’s local neighborhood, rather than optimizing a separate embedding vector for each node directly. A stack of `SAGEConv` layers implements this idea as full-batch message passing over a split’s observed edges: rather than the mini-batch neighbor-sampling procedure of the original GraphSAGE formulation, every layer aggregates over the complete set of in-neighbors present in the split’s edge index. At depth k , a single `SAGEConv` layer updates every node v in two steps:

- it aggregates the previous-layer representations of all of v ’s neighbors into a single vector using mean aggregation, a permutation-invariant function.
- it concatenates this aggregated neighborhood vector with v ’s own representation from the previous layer and passes the result through a linear transformation; on every layer except the last, this is followed by a ReLU non-linearity and dropout.

Stacking K such layers, with each node’s representation initialized to its input feature vector, lets a node’s final embedding depend on its K -hop neighborhood, progressively folding in information from further away at each additional layer. Crucially, because what is learned is the aggregation function itself rather than a lookup table of fixed embeddings, the same trained layers can be applied to nodes that were never seen during training, as long as they carry features and known edges: this is what makes the encoder inductive rather than transductive, and it is the property that lets the same trained model be evaluated on the `dev` and `test` splits without any further gradient updates.

Given a candidate pair (u, v) , a decoder scores the pair from the concatenation $[h_u || h_v]$ passed through a small MLP. This concatenation is a deliberate choice over a symmetric dot product $h_u \cdot h_v$: a dot product is invariant to swapping u and v and therefore cannot express directionality, whereas the task is specifically "does $u \rightarrow v$ hold", not "are u and v connected". Negative pairs are sampled as directed non-edges, so a pair (u, v) only counts as negative when $u \rightarrow v$ does not exist, even if the reverse edge $v \rightarrow u$ does, otherwise the model could learn to detect undirected connectivity rather than direction.

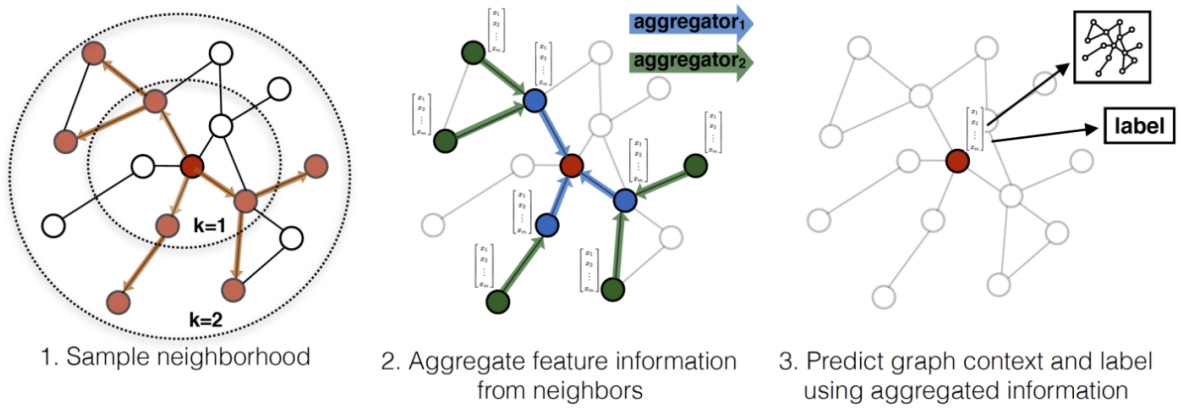


Figure 4: Visual illustration of the GraphSAGE sample and aggregate approach.

4.4.7 SEAL

The third model is SEAL [8]: rather than scoring a pair from two independently-computed node embeddings, SEAL extracts the local enclosing subgraph around $\{u, v\}$, labels every node in it by its structural position relative to u and v via Double-Radius Node Labeling (DRNL), and runs a GNN with sort-pooling (DGCNN) over that labeled subgraph to produce a single pair score. This gives the model access to the fine-grained local topology around a candidate pair, rather than only the two endpoint embeddings. The original formulation of SEAL assumes an undirected graph, and was adapted here in three ways to make it direction-aware:

- DRNL distances are computed with direction: the distance from u is measured by walking forward along citation edges, and the distance to v by walking backward from v . A node reachable only downstream of u therefore receives a different label than one reachable only upstream of v , encoding exactly the asymmetry the task requires.
- The enclosing subgraph itself keeps its edges directed rather than symmetrized, so the GNN’s message passing also respects citation direction. Node membership in the subgraph is still gathered via undirected reachability, so a node connected to u or v through either direction can be included, but once selected, only its real directed edges are kept.
- Only the exact directed edge $u \rightarrow v$ under test is removed before extraction and labeling.

A genuine reverse edge $v \rightarrow u$, if present, is left in place, since it is legitimate structural context rather than a leak of the label being predicted.

Each subgraph node carries the same content-plus-structural feature vector used by the other two models: its 768-dimensional SPECTER2 embedding concatenated with log-scaled in-degree, log-scaled out-degree, and the seed/expansion flag, so the final score can draw on paper content and global citation profile in addition to the pair’s local topology. Because extracting and scoring one subgraph per candidate pair is considerably more expensive than a single forward pass through a shared encoder, both subgraph size and hop radius are capped (a 1-hop neighborhood, trimmed to at most 100 nodes per subgraph), and pairs are subsampled rather than exhaustively enumerated (20,000 training pairs, 4,000 evaluation pairs).

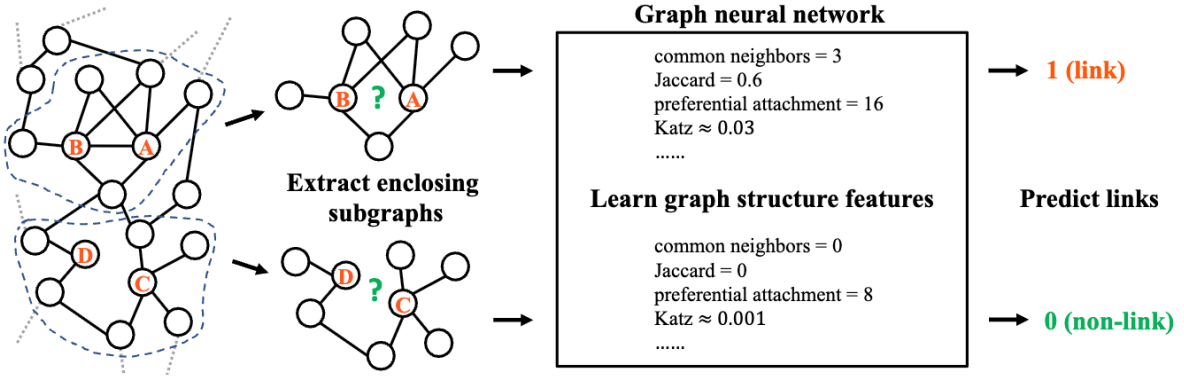


Figure 5: Visual representation of the original SEAL framework.

4.4.8 Web application

To make the link-existence prediction method usable by an actual researcher rather than existing only as an offline evaluation number, a standalone web application was built on top of it: given a paper, it surfaces other papers the model considers plausible future citations, explains why for each one, and lets the researcher explore the resulting subgraph interactively, deliberately modeled on tools like Research Rabbit, but with the explainability those tools lack as we have seen in Section 2.

The webapp is built around whichever graph-based link predictor was selected as best-performing among the models introduced above, rather than around a specific architecture: the interface, the explanation pipeline, and the bucket logic described below only assume that the underlying model exposes (i) a per-paper embedding and (ii) a pairwise decoder that scores a candidate pair from those embeddings, a contract satisfied by any of the trained models in this project. All of the model-specific computation, embedding a paper and running it through the decoder, is done offline: a precomputation script loads the trained checkpoint once, computes an embedding for every paper in the evaluation split, and exports both the embeddings and the decoder’s weights as small files. The webapp itself never loads the graph, the model training framework, or the checkpoint directly; at request time it is a lightweight process that re-implements the decoder’s forward pass in plain array arithmetic over the precomputed embeddings, so serving a request is fast and has no heavy dependency on the training stack.

Given a query paper, the webapp does not score it against every other paper in the corpus, since that would not scale. It first shortlists a few hundred candidates by embedding cosine similarity, a cheap operation, and only then re-scores that shortlist exactly with the trained decoder, a more expensive but far more accurate step. This two-stage retrieve-then-rerank pattern lets the interface stay responsive while still using the model’s own decision boundary, not just embedding

proximity, to score each candidate. Papers the query already cites are excluded from the shortlist, since the goal is to surface plausible *new* connections, not to rediscover known citations.

Rather than presenting the re-scored candidates as one flat ranked list, which tends to always surface the same highly-cited, well-established papers, the webapp splits them into three views: an *all-time greats* bucket (candidates with high citation-graph degree within the indexed subgraph, i.e. well-established work), a *promising stars* bucket (candidates published recently, judged against the real calendar date rather than against other candidates in the shortlist, with the recency window widened adaptively when a query’s own topical neighborhood has nothing that recent), and a *hidden gems* bucket (everything else: older, low-degree, but still flagged relevant by the model despite never having caught on). The first split is by rank within the shortlist rather than a fixed threshold, since the indexed subgraph’s degree distribution is extremely sparse and a fixed threshold would degenerate into a single non-empty bucket; the second is a genuine absolute date cutoff, so a candidate is never mislabeled "promising" purely because it happens to be the least old option available. Within each bucket, candidates are ordered by topical relevance, cosine similarity in the raw SPECTER2 content-embedding space, distinct from the GraphSAGE-embedding similarity used for shortlisting, rather than by the decoder’s predicted probability, since that probability saturates near the top of a shortlist and stops discriminating meaningfully between candidates on topic; predicted probability is kept only as a tie-breaker, and as the sole ranking key when a precomputed index predates the topical-relevance field.

Alongside the buckets, the webapp renders a subgraph centered on the query paper: its real citation neighbors, both papers it cites and papers that cite it, drawn from the same citation-graph index used elsewhere in the project, plus the model’s own predicted links drawn into the same view, visually distinguished from real edges. A predicted edge is only drawn when the model’s score clears a confidence threshold, so the graph never implies the model endorses a suggestion it is actually unsure about; below that threshold, a candidate can still appear ranked in a bucket with its real, low score shown, just not as an implied prediction. Clicking any node in the graph re-centers the view on that paper, letting the researcher roam the neighborhood rather than being confined to a single query.

Every suggested candidate, in either the buckets or the graph, comes with an explanation of why the model scored it highly, addressing the explainability gap identified in Section 2: a Symbolic Knowledge Extraction (SKE) decision tree, fit offline on a sample of real edges and non-edges to approximate the trained decoder’s output from five interpretable features (embedding cosine similarity, number of shared citation neighbors, publication year gap, candidate citation-graph degree and topical relevance, the raw SPECTER2 content-embedding cosine similarity introduced above), is queried first, since it is cheap and gives an actual human-readable rule. When the tree’s global fidelity is too low to be trusted at all, or its prediction for this specific candidate disagrees with what the real model actually output, the webapp instead computes a live, per-instance LIME explanation: the candidate’s real embedding is perturbed with Gaussian noise, the real trained decoder scores every perturbed sample, and a local, proximity-weighted linear surrogate is fit on the same five features, trading the tree’s cheap global rule for a slower but locally faithful one.

4.5 Second-hand citation detection

Beyond predicting whether a citation edge exists, the enriched citation graph also supports a separate, self-contained task: second-hand citation detection, spotting cases where a paper u cites a paper x not because its authors engaged with x directly, but because they picked it up indirectly through an intermediate paper w that they read and that itself cites x . This task does not build on, retrain, or query any of the trained link-existence models (MLP baseline, GraphSAGE, SEAL) described above: it operates directly on the citation graph’s structure and on the citing sentences already attached to each edge, combined with a single off-the-shelf

pretrained language model used purely to embed text. No model here is trained on this project’s data; the only learned component is a frozen, pretrained sentence encoder used as a fixed feature extractor.

4.5.1 Closed triads and the role of SciBERT

A candidate for this pattern is a closed triad (u, w, x) : $u \rightarrow w$, $w \rightarrow x$, and $u \rightarrow x$ are all real edges, found by a direct structural query over the graph. The triad shape alone is not sufficient evidence, though: the same structural pattern arises whenever a well-cited paper x happens to share citers with w for entirely unrelated reasons, so structure can only narrow down candidates, not confirm the second-hand relationship itself.

To go beyond structure, the task uses the citing sentence, the actual sentence in u where x is cited, and compares it to the citing sentence w uses for x . If u picked up x through w , its citing sentence is expected to echo w ’s framing of x more closely than two authors who each arrived at x independently normally would. This is where SciBERT enters: it is a pretrained language model, used unmodified and without any task-specific fine-tuning, whose sole role here is to turn each citing sentence into a fixed-size embedding that captures its semantic content. It is not a graph model and does not see the citation graph at all; it only ever sees text, one citing sentence at a time. Only the sentences actually needed for the triads under evaluation are embedded, rather than the full citation-context corpus, since embedding is the most expensive step in the pipeline and most citing sentences in the corpus never participate in a candidate triad.

4.5.2 A per-target null model and the role of percentiles

A raw similarity score between two citing sentences is not, by itself, informative: some target papers x naturally attract citers who describe them in very similar terms regardless of any second-hand relationship, for instance because the paper introduces a specific named method or result that every citer paraphrases the same way. Judging a similarity score in isolation would flag those common papers constantly, and it would take a genuinely unusual similarity for other papers as unremarkable.

The task corrects for this with a per-target null model: for each target x , the citing-sentence similarity between every other pair of x ’s citers, excluding the specific (u, w) pair under test, is computed and collected into an empirical background distribution, in effect, "how similar do two independent citers of x normally sound." A triad’s score is then not the raw similarity itself but its percentile rank within that background: a value close to 1.0 means u and w ’s citing sentences for x are more alike than nearly all other citer pairs of x , which is the evidence taken as suggestive of a second-hand relationship, calibrated specifically to what is normal for that target rather than to a single global threshold. To keep this background tractable for heavily-cited targets, where the number of citer pairs grows quadratically, the number of background pairs considered per target is capped and randomly subsampled.

4.5.3 Temporal plausibility

A closed triad only tells a coherent " u picked up x through w " story if the publication order is consistent with it: x must have been published before w , and w before u . This is checked explicitly rather than assumed, since the structural correction pass of Section 4.3.5 only removes impossible length-2 reciprocal cycles and does not guarantee every longer path in the graph is temporally coherent.

The check uses each paper’s publication year as a first filter, refined by day-level publication date wherever it is available and parseable, to also catch same-year orderings that year alone cannot distinguish. Triads that fail either check are discarded before scoring, so the textual-similarity

and percentile steps above are only ever run on candidates for which the second-hand story is structurally and temporally possible in the first place.

4.5.4 Web application

Since the second-hand citation task above produces a scored list of flagged triads rather than a single yes/no verdict, a separate web application makes those results browsable and inspectable, rather than requiring anyone to read them out of a raw results table.

All of the expensive computation, the SciBERT embedding and the per-target percentile scoring described above, is done entirely offline, in advance, by the analysis script itself. The webapp never loads SciBERT, PyTorch, or any embedding model at request time; it only reads the resulting table of scored triads, one row per flagged (u, w, x) candidate with its similarity score, its percentile rank against that target’s background, and the publication years and citing-sentence text for all three papers. Because this table already contains every value the interface needs, the application can stay a lightweight process with no heavy machine-learning dependency in its serving path, mirroring the same offline-precompute, lightweight-serve split used by the link predictor webapp in the previous section.

The default view is a global, filterable list of flagged triads, rather than requiring the user to already know which paper they are interested in: results can be filtered by a minimum percentile threshold, sorted, and searched by free text over paper titles and citing sentences. Each result is rendered as a three-part card, source paper, intermediate paper, and citing paper, laid out in that order with the two citing sentences shown side by side, so the textual echo the percentile score is meant to capture is directly visible rather than only asserted by a number. When the citer (U) and the intermediate paper (W) share at least one author – loosely matched by lowercased, whitespace-trimmed name string against the metadata cache, since only the raw author-name strings, not disambiguated author IDs, are available – the card additionally carries a *shared author(s)* badge, and the explanation panel names the overlapping author(s) with a note that the textual echo between U’s and W’s citing sentences is at least as plausibly the same author(s) reusing their own prior phrasing as it is a genuine second-hand pickup through an unrelated paper. This case is flagged rather than filtered out, for the same reason the tool surfaces every candidate it finds: a same-author overlap is a real, imperfect confound on the second-hand-pickup interpretation specifically, not evidence that the underlying textual echo itself is spurious, so the decision of how much weight to give a same-author flag is left to the researcher rather than made silently by the tool.

Each flagged result also has a dedicated graph view, built specifically around that one triad rather than as a general citation-neighborhood explorer: it always shows exactly the three papers u , w , and x , colored and labeled by their role in the triad, with the three triad edges ($u \rightarrow w$, $w \rightarrow x$, $u \rightarrow x$) drawn as highlighted, solid edges distinct from everything else in the view, so the second-hand-citation pattern itself is visually legible at a glance rather than requiring the reader to mentally reconstruct it from three separate node identities. A small, capped amount of additional citation context, a few extra neighbors around each triad paper, drawn from the same citation-graph index used by the link predictor webapp, can be layered in as faint, secondary edges, giving the researcher a sense of the triad’s surrounding neighborhood without turning the view into an open-ended exploration tool; that broader kind of exploration is deliberately left to the link predictor webapp described earlier, so each application keeps a distinct, single purpose.

5 Results

In this chapter, the results of the models presented in Section 4.4 and the results of the second-hand citation tool presented in Section 4.5 are presented and discussed.

5.1 Link-existence prediction

The graph-based methods for link prediction are presented here. As mentioned previously in Section 4.4, the models were fit on both randomly sampled pairings and more realistic ones, the latter verifying the possible presence of a link between nodes that are close in the graph (i.e. at a distance of 2 hops) rather than between arbitrary, unrelated papers.

Since random-pairing results systematically overvalue the models' real performance, only the results on the more realistic, hard-negative pairings are presented in this report. The corresponding overconfident random-pairing results can instead be found in the notebook `train_and_test_NEW.ipynb`, for reference.

5.1.1 Best hyperparameter configurations

Table 3 reports the best hyperparameter configuration found for each of the four models as described in Section 4.4.2's hard-negative variant in every case. The models were selected by validation AP during the grid search described in Section 4.4. The "raw" version of the MLP baseline has no tunable layer count, since it has no per-node encoder at all: its hidden dimension applies only to the shared pairwise decoder. SEAL has no weight-decay column; its DRNL label-embedding dimension is tuned in its place, as described in Section 4.4.

Model	Learning rate	Hidden dim.	Layers	Weight decay / label-dim	Val. AP
MLP baseline (raw)	10^{-2}	256	–	5×10^{-4}	0.8758
MLP baseline (encoded)	10^{-3}	64	3	5×10^{-4}	0.8654
GraphSAGE	10^{-3}	128	3	5×10^{-4}	0.8816
SEAL	10^{-3}	32	–	label-dim = 16	0.8716

Table 3: Best hyperparameter configuration per model (hard-negative variant), selected by validation AP during grid search.

Note that these validation AP values, taken during hyperparameter search, are not directly comparable to the final test AP reported in Section 5.1.2: the reported test numbers come from a separate final training run using this configuration, not from the tuning trial itself. However very similar results were obtained, resulting in no loss of generalization.

5.1.2 Test results

Table 4 reports the final test AUC and test AP for each model, using the hyperparameter configuration in Table 3, evaluated once on the held-out `test` split as described in Section 4.4.3.

Model	Test AUC	Test AP
MLP baseline (raw)	0.9290	0.8973
MLP baseline (encoded)	0.9218	0.8894
GraphSAGE	0.9273	0.8897
SEAL	0.9251	0.9130

Table 4: Test-set AUC and AP for each model, hard-negative variant.

The four models are close on AUC, within roughly half a percentage point of each other, but SEAL shows a clearer advantage on AP, the metric prioritized throughout this project as shown in Section 4.4.3. This result suggests it does better specifically at ranking true positive edges above hard negatives rather than only separating the two classes on average. Notably, the raw MLP baseline, with no learned per-node transformation and no graph structure at all,

outperforms GraphSAGE on AP while remaining close to it on AUC, and the encoded MLP variant does not clearly improve on it, indicating most of the achievable performance on this hard-negative task may already be captured by the node features themselves rather than by the encoder architecture on top of them.

5.1.3 Web application

Despite SEAL achieving the highest test AP among the four models (Table 4), the link predictor webapp described in Section 4.4 deploys GraphSAGE rather than SEAL. This was a deliberate decision made on serving-time grounds rather than on raw test-metric ranking alone.

The two models differ fundamentally in how a candidate pair gets scored, not just in accuracy. GraphSAGE produces a single fixed embedding per paper, computed once, offline; scoring any candidate pair afterward is a cheap, vectorized forward pass through the decoder over precomputed embeddings, with no dependency on `torch` or the citation graph itself at request time. SEAL has no equivalent precomputation step: its score for a pair only exists after extracting and running a GNN over that specific pair’s own enclosing subgraph, an operation repeated for every candidate at request time rather than something that can be computed once and reused. Since the webapp reranks a shortlist of a few hundred candidates per query (Section 4.4), scoring all of them with SEAL live would mean hundreds of subgraph extractions and GNN forward passes per request, and would reintroduce `torch` and the full citation graph as a serving-time dependency, undoing the lightweight, offline-precompute design the webapp was built around. GraphSAGE’s roughly 2.3-point lower AP was judged an acceptable cost for keeping the interface interactive and dependency-light, given how much more expensive SEAL would be to serve at the same scale.

Figure 6 walks through one concrete example, using the same pipeline described in Section 4.4, to make the abstract description above concrete. The query paper is “Beneficial effects of inhibition of angiotensin-converting enzyme on ischemic myocardium during coronary hypoperfusion in dogs” (Kitakaze et al., 1995, *Circulation*), one of the curated example papers surfaced on the webapp’s landing page. Searching for this paper and opening it shows its citation subgraph on the left and the three bucketed lenses on the right, each ranked internally by topical relevance. The top candidate in the *all-time greats* bucket, “The role of cardiovascular magnetic resonance imaging in heart failure” (Karamitsos et al., 2009, *J. Am. Coll. Cardiol.*), has a predicted probability of 0.987, a cosine similarity of 0.997, a topical relevance of 0.904, and a citation-graph degree of 142 within the indexed subgraph. Clicking “Explain this prediction” shows that the global decision-tree surrogate is trusted for this instance (its prediction of 0.938 is within the disagreement tolerance of the model’s real 0.987, and its overall fidelity is $R^2 = 0.967$), so the tree’s own decision path is shown directly rather than falling back to LIME: the path splits three times on `candidate_degree` (this candidate’s degree of 142 clears all three thresholds, at 9.5, 19.5, and 26.5) and once on `cosine_similarity` (this candidate’s 0.997 clears a threshold of only 0.149).

The other two buckets for this same query surface candidates that are genuinely on-topic as well: the *promising stars* top candidate, “Review of a causal role of fructose-containing sugars in myocardial susceptibility to ischemia/reperfusion injury” (Maarman et al., 2017, *Nutrition Research*), and the *hidden gems* top candidate, “Cardiac interstitial bradykinin release during ischemia is enhanced by ischemic preconditioning” (Pan et al., 2000, *Am. J. Physiol. Heart Circ. Physiol.*), both concern ischemia directly, unlike the off-topic suggestions a purely probability-sorted list would have surfaced instead.

Ranking within each bucket by topical relevance rather than predicted probability is what produces this: for the *promising stars* candidate, the tree’s prediction of 0.751 falls outside the disagreement tolerance of the model’s real 0.988, so this suggestion is explained by LIME instead of the tree.

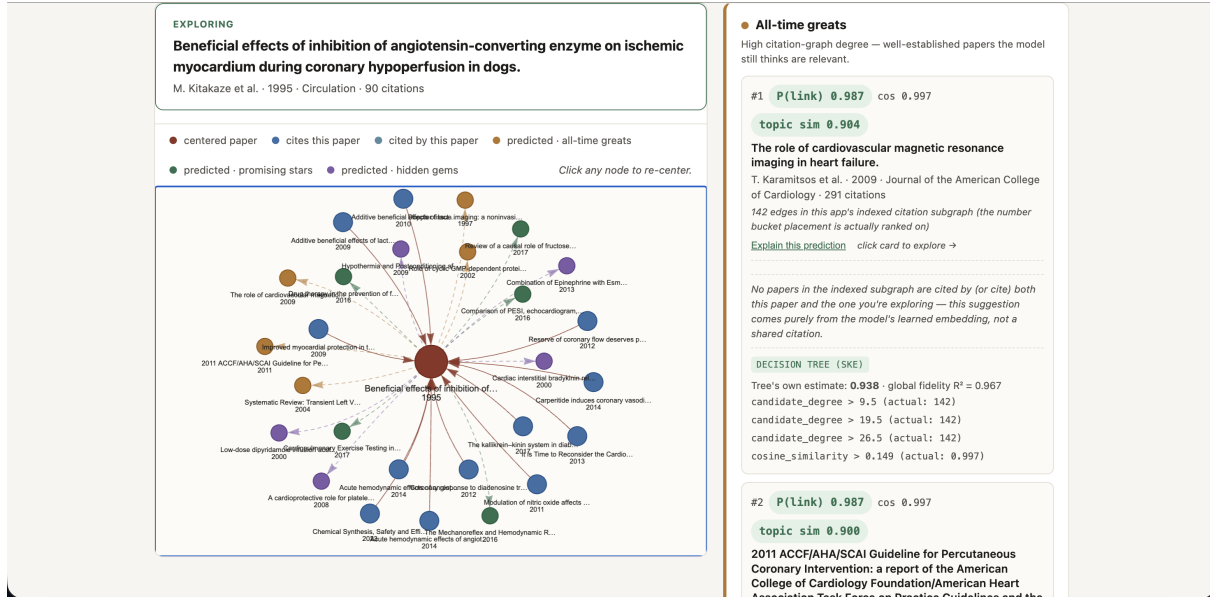


Figure 6: The webapp’s explore view for the example query paper, showing the subgraph, the three bucketed lenses, and the decision-tree explanation for the top-ranked candidate.

Table 5 lists these three top candidates together, without requiring a separate screenshot for each: since the interface and interaction pattern shown in Figure 6 is identical across buckets, differing only in which candidates populate the panel, the underlying numbers are given directly instead.

Bucket	Top candidate	$P(\text{link})$	Topical rel.	Degree	Year
All-time greats	“The role of cardiovascular magnetic resonance imaging in heart failure” (Karamitsos et al., 2009, <i>J. Am. Coll. Cardiol.</i>)	0.987	0.904	142	2009
Promising stars	“Review of a causal role of fructose-containing sugars in myocardial susceptibility to ischemia/reperfusion injury” (Maarman et al., 2017, <i>Nutrition Research</i>)	0.988	0.901	11	2017
Hidden gems	“Cardiac interstitial bradykinin release during ischemia is enhanced by ischemic preconditioning” (Pan et al., 2000, <i>Am. J. Physiol. Heart Circ. Physiol.</i>)	0.990	0.923	72	2000

Table 5: Top-ranked candidate from each of the three buckets for the same query paper, ranked by topical relevance within each bucket.

Additional examples can be explored directly in the webapp; setup and usage instructions are provided in the repository’s README.md.

5.2 Second-hand citation detection

The second-hand citation detector described in Section 4.5 was run on the `test` split with the same defaults used elsewhere in this project: closed-triad search capped at 5,000 candidates, a

target paper needs at least 3 citers to have a usable background, and each target’s background is capped at 200 randomly subsampled citer pairs. Of the candidate closed triads found, 219 survived every filtering step, structural closure, an available and sufficiently large background for the target, non-empty citing-sentence text on both sides, and the temporal plausibility check, and were scored. Table 6 summarizes the resulting percentile distribution. Percentile scores span the full range, with a median close to the center of the distribution (0.581) rather than clustering near either extreme, which is what would be expected if genuinely suspicious triads are the exception rather than the rule: most closed triads look like ordinary, independent citations of the same target, and only a minority stand out as textually unusual.

Statistic	Value
Scored triads	219
Mean percentile	0.576
Median percentile	0.581
Triads with percentile ≥ 0.80	67 (30.6%)
Triads with percentile ≥ 0.90	38 (17.4%)
Triads with percentile ≥ 0.95	17 (7.8%)
Triads with percentile ≥ 0.99	12 (5.5%)

Table 6: Distribution of percentile scores across the 219 scored triads, test split.

Figure 7 walks through the single highest-scoring triad, which reaches a percentile of 1.00: paper U, “Into the depths of spatial attention and multisensory integration” (Van der Stoep et al., 2015), cites paper X, “Spatial cuing in a stereoscopic display: Evidence for a ‘depth-aware’ attentional focus” (Atchley et al., 1997), with the sentence “Our findings are in line with the results from studies in which exogenous intramodal orienting of attention was investigated in 3-D space (Atchley et al., 1997; Theeuwes & Pratt, 2003).” The intermediate paper W, “Exogenous orienting of crossmodal attention in 3-D space: Support for a depth-aware crossmodal attentional system” (Van der Stoep et al., 2013), cites the same paper X with the *identical* sentence, word for word. Figure 8 shows the corresponding triad graph, opened via the “View triad graph” button on that same card.

This example is worth being precise about, since the identical wording is stronger evidence of *textual reuse* than of the specific second-hand-citation story the method is trying to detect. U and W share all three of their authors, in this case the same author list in a different order, so an equally plausible explanation is that the same research group reused their own citing sentence verbatim across two of their own papers, rather than U’s authors having picked up X’s framing indirectly by reading W. The method as built cannot distinguish these two cases on its own: it detects an unusually strong textual echo between a paper’s citing sentence and an earlier paper’s citing sentence for the same target, but common authorship is a confound it does not control for by construction.

This is why every flagged card in the webapp carries a same-author check alongside the percentile score: for each result, the citer’s (U) and the intermediate paper’s (W) author lists are loosely matched by name string, and any overlap is surfaced directly on the card as a *shared author(s)* badge, with the overlapping names and a caveat named in the explanation panel below it. For this example the badge fires and names all three shared authors, making the self-reuse explanation visible to the reviewer at the point they see the result rather than requiring them to notice it themselves or look it up separately.

The flag is displayed rather than used to filter the triad out of the results: a same-author overlap is a real, imperfect confound on which of the two explanations is correct, not evidence that the underlying textual echo itself is spurious, so this case is still shown, with the confound made explicit, rather than hidden.

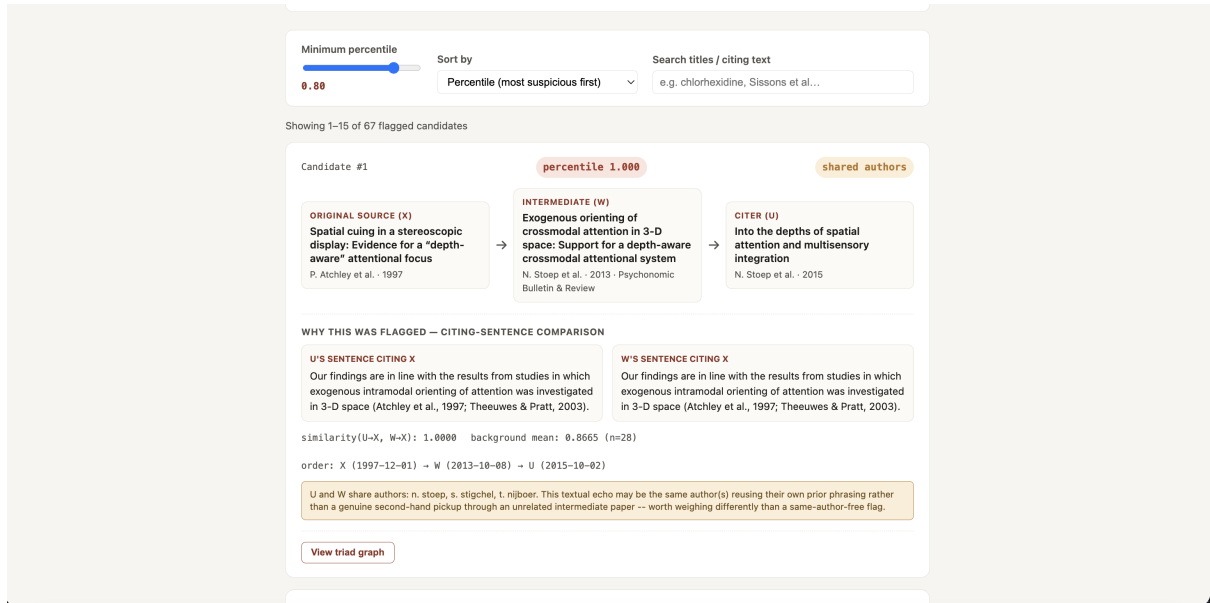


Figure 7: The second-hand citation webapp's flagged-candidate card for the highest-scoring triad (percentile 1.00), showing the U/W/X chain, the side-by-side citing-sentence comparison, and the shared author(s) badge.

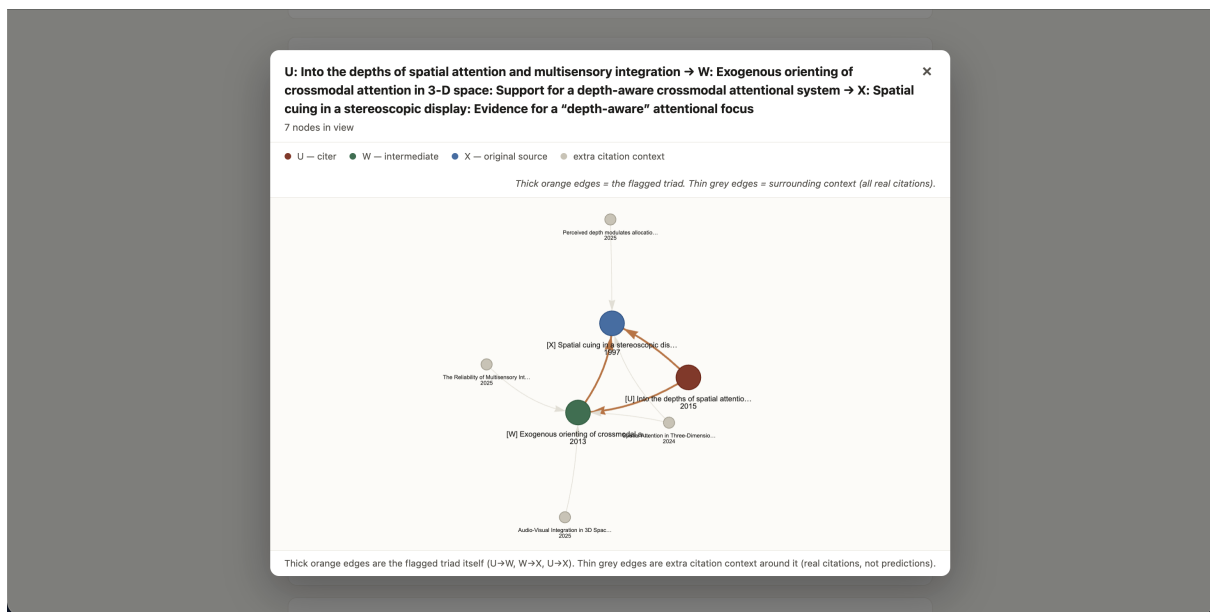


Figure 8: The triad graph for the same flagged candidate: U, W, and X colored and labeled by role, with the triad edges highlighted (thick orange) against a small amount of real citation context (thin grey).

Other examples can be seen by navigating the webapp, the instructions to do this are in the README.md file in the repository.

6 Conclusions

CitGraph sets out to treat a citation as something to characterise and audit rather than as an interchangeable unit to count. Starting from SciCite's flat, sparse citation instances, the project

built a pipeline that expands, enriches, and temporally corrects a citation graph, and used it to support two distinct tasks: predicting whether a citation link plausibly exists, and flagging citations that may have been picked up second hand rather than read directly.

On the first task, three model families were compared under a shared, realistic evaluation protocol built on structurally plausible hard negatives rather than easy random ones, with SEAL reaching the best test AP of 0.9130 while GraphSAGE was deployed in the web application for serving cost reasons made explicit in the results section.

On the second task, a lightweight, entirely offline pipeline combining closed triad structure, a frozen SciBERT encoder, and a per target null model surfaced a small, plausible set of second hand citation candidates, 219 out of an initial pool capped at 5000, with the single highest scoring example illustrating both the method’s sensitivity.

Both tasks were packaged into interactive, explainable web applications rather than left as offline evaluation numbers, directly answering the explainability gap identified in the related work discussion. This was done not only to show a possible usage of the work which was presented, but also to integrate the blind spots which these two methods had (i.e. the shared authorship and the saturation of the cosine similarity for the GraphSAGE method). This will be further discussed in Section 6.1.

The section below lays out the limitations of the design and the concrete steps to improve it in the future.

6.1 Limitations

This section separates two kinds of limitation. The first group covers implementation level gaps specific to the current webapp behaviour. The second covers structural limitations in how the whole project would need to change to handle a substantially larger graph or an actual production deployment.

6.1.1 Implementation level limitations

Degree can dominate over relevance in the explanation. As shown by the cardiology query, when candidates cluster tightly on cosine similarity, the SKE tree falls back to candidate degree as its main discriminator, surfacing popular but topically unrelated papers with high confidence. The buckets make this visible rather than hidden, however this was tackled by introducing a reranking based on topical relevance, solving partially this problem.

Bucket membership ignores the prediction confidence threshold. The threshold that decides whether a predicted edge is drawn in the graph view is not applied when assigning a candidate to a bucket, so a low confidence candidate can still rank at the top of hidden gems, however the prediction probability of the link and the topical relevance are stated clearly in order to inform the user about the low confidence of the model for a given link.

Second hand citation detection cannot separate self reuse from genuine second hand reading. The highest scoring triad discussed in Section 5.2 is at least as well explained by shared authorship between the citing and intermediate paper as by an indirect citation pickup. The design choice for this limitation was the same as for the previous one: noticing about the shared authorship, in order to let the user take an informed choice.

The deployed link predictor is not the best performing one. GraphSAGE was chosen over SEAL for the web application purely on serving cost grounds. This is a reasonable engineering tradeoff, but it means the production system runs roughly 2.3 AP points below what the project’s own evaluation shows is achievable.

6.1.2 Scale and production readiness

Every design choice made throughout the project is calibrated to a graph of a few hundred thousand nodes, the size the final SciCite derived splits happen to land at after expansion. Several of these choices do not degrade gracefully once that assumption is pushed even one order of magnitude further, they stop working outright, and none of this was exercised empirically. Everything below is read off the pipeline’s own design, not measured against an actually larger graph.

The encoder training loop assumes the whole split fits in one machine’s RAM and one GPU’s memory. GraphSAGE and the MLP baselines run full batch message passing over an entire split’s edge index in a single forward pass. The graph loading function deserializes the whole cached graph from one pickle file, and the offline recommender index build step states its own requirement explicitly. It needs enough RAM to hold the full graph, several gigabytes on disk for this project’s train split alone, before a single embedding is computed. Neither the encoder nor the loading path has any notion of sharding a graph across machines or streaming it in pieces. Both approaches assume the graph is small enough to sit in one process’s memory for the entire run. The decoder batch size option only chunks the decoder’s pairwise scoring step, never the encoder’s message passing pass itself, so it does not relieve this constraint.

SEAL’s per pair subgraph extraction does not scale sublinearly with candidate count. Each candidate pair requires its own breadth first search based neighbourhood query and subgraph induction, which is why training and evaluation pairs are explicitly capped and subsampled at 20000 and 4000 respectively rather than exhaustively enumerated. This cap is a direct concession to cost, not a modelling choice. On a graph with substantially higher average degree, the same one hop, hundred node capped extraction would either need a tighter cap, losing structural signal, or become the dominant cost of the whole pipeline.

Hard negative mining is computed once, per split, over every node, and held fully in memory. The two hop target precomputation step materializes the two hop out neighbourhood of every node in the graph as a single in memory dictionary before training starts. This is linear in node count only under the assumption that per node neighbourhoods stay small. A graph with a denser hub structure, a small number of extremely highly cited papers, which is exactly the kind of skew real citation graphs are known for, would make individual neighbourhoods, and therefore this precomputation step, far more expensive than the node count alone suggests.

The deployed webapp is a single process, single machine service, not a production deployment. The link predictor webapp runs Flask’s built in development server directly with debug mode enabled, holds its entire recommender index as plain in memory arrays behind a lazily initialized global, and answers every query with a fresh exact cosine similarity pass over all indexed papers rather than an approximate nearest neighbour index. This is an entirely reasonable choice for a project scale demonstrator serving requests one at a time against a few hundred thousand papers, but none of it is a production serving architecture. The retrieve then rerank shortlist step would need a real vector index rather than a linear scan the moment the corpus grows past what fits comfortably in one process’s memory.

Model and hyperparameter choices were validated at this scale only. The grid search tunes learning rate, hidden dimension, and depth within ranges chosen for graphs of this size. Nothing in the tuning setup or the final architecture choices was tested against how these hyperparameters, or the relative ranking between GraphSAGE, SEAL, and the MLP baselines, would hold up on a graph an order of magnitude larger, where the tradeoffs between full batch message passing, per pair subgraph extraction, and a graph free baseline could shift substantially.

6.2 Future work

The limitations above point to concrete next steps, several of which reuse data or infrastructure the project already has.

6.2.1 Addressing the implementation level limitations

Evaluating the proposed GraphSAGE and SEAL hybrid is a clear next experiment. Re scoring only the small, final shortlist actually shown to the user with SEAL, after GraphSAGE handles the cheap large shortlist rerank as it does now. Doing so would show whether SEAL’s AP advantage concentrates on exactly the ambiguous cases where getting the ranking right matters most, while allowing to use the best performing model in the webapp.

It would be interesting to test the link predictors against adversarially perturbed embeddings or citing sentences, rather than only structurally plausible hard negatives. This would extend the current hard negative training, leading to a more robust model for everyday usage.

Revisiting the length three cycle edges left uncorrected would close the one acknowledged gap in an otherwise systematic correction pass. These cycles were deliberately left untouched as out of scope, and a case by case or heuristic resolution for them, a small minority of the paradoxical cycles found, is a reasonable next step.

6.2.2 Addressing scale and production readiness

Moving from full batch to mini batch message passing would remove the single largest structural blocker identified above. Adopting the neighbour sampling mini batch procedure from GraphSAGE’s original formulation, rather than this project’s full batch variant, would decouple training memory from split size and let the encoder scale past what fits in one GPU or one machine’s RAM.

Sharding or streaming the graph loading path instead of relying on one monolithic pickle file would remove the RAM ceiling on both training and the offline recommender index build step. Replacing the current single file, whole graph in RAM load with a partitioned or on disk indexed representation, in the spirit of the SQLite backed enrichment cache already used elsewhere in the pipeline, is a direct way to achieve this.

Replacing SEAL’s per pair extraction cap with a cheaper approximate subgraph summary, or restricting it to the reranking stage only, follows naturally from the hybrid pipeline proposed above. Since that hybrid already limits SEAL to a small final shortlist, the same restriction is also the most direct way to keep its per pair cost bounded on a much larger graph without lowering the current subgraph size cap further.

Bounding hard negative precomputation against hub degree explicitly would keep this step’s cost predictable. Extending the existing per node cap with an early cutoff on raw neighbourhood size before materializing it, mirroring the hub aware sampling already used for candidate generation elsewhere in the same module, would keep this step’s cost proportional to the cap rather than to the graph’s real degree distribution.

Introducing an approximate nearest neighbour index for the retrieval stage would let the system scale to a much larger corpus. Replacing the webapp’s exact, linear scan cosine similarity shortlist with an index built for this purpose would let the retrieve then rerank pattern keep its current latency characteristics on a corpus far larger than what an in memory linear scan can serve per request.

Moving the webapp behind a production grade serving stack is a prerequisite independent of any model or algorithm change. Running the application behind a proper application server rather than the built in development server, disabling debug mode, and adding a persistence layer for the precomputed index would be the minimum needed for any deployment beyond a local demonstrator.

Re-running the hyperparameter search and model comparison at a larger scale would close the validation gap named above. Since the ranking between GraphSAGE, SEAL, and the MLP baselines was only ever validated at this project’s graph size, repeating the grid search on a substantially larger graph, once the scaling changes above make that feasible, would confirm whether the same architecture and hyperparameter choices still hold, or whether a different

tradeoff point emerges at scale.

References

- [1] Fabian Beck. Puresuggest: citation-based literature search and visual exploration with keyword-controlled rankings. *IEEE Transactions on Visualization and Computer Graphics*, 31(1):316–326, 2024.
- [2] Dan Berrebbi, Nicolas Huynh, and Oana Balalau. Graphcite: Citation intent classification in scientific publications via graph embeddings. In *Companion proceedings of the web conference 2022*, pages 779–783, 2022.
- [3] Arman Cohan, Waleed Ammar, Madeleine Van Zuylen, and Field Cady. Structural scaffolds for citation intent classification in scientific publications. In *NAACL*, 2019.
- [4] Arman Cohan, Sergey Feldman, Iz Beltagy, Doug Downey, and Daniel S. Weld. SPECTER: Document-level Representation Learning using Citation-informed Transformers. In *ACL*, 2020.
- [5] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs, 2018.
- [6] Rodney Michael Kinney, Chloe Anastasiades, Russell Authur, Iz Beltagy, Jonathan Bragg, Alexandra Buraczynski, Isabel Cachola, Stefan Candra, Yoganand Chandrasekhar, Arman Cohan, Miles Crawford, Doug Downey, Jason Dunkelberger, Oren Etzioni, Rob Evans, Sergey Feldman, Joseph Gorney, David W. Graham, F.Q. Hu, Regan Huff, Daniel King, Sebastian Kohlmeier, Bailey Kuehl, Michael Langan, Daniel Lin, Haokun Liu, Kyle Lo, Jaron Lochner, Kelsey MacMillan, Tyler C. Murray, Christopher Newell, Smita Rao, Shaurya Rohatgi, Paul Sayre, Shannon Zejiang Shen, Amanpreet Singh, Luca Soldaini, Shivashankar Subramanian, A. Tanaka, Alex D Wade, Linda M. Wagner, Lucy Lu Wang, Christopher Wilhelm, Caroline Wu, Jiangjiang Yang, Angele Zamarron, Madeleine van Zuylen, and Daniel S. Weld. The semantic scholar open data platform. *ArXiv*, abs/2301.10140, 2023.
- [7] MV Simkin and VP Roychowdhury. Theory of citing. In *Handbook of Optimization in Complex Networks: Theory and Applications*, pages 463–505. Springer, 2011.
- [8] Muhan Zhang and Yixin Chen. Link prediction based on graph neural networks, 2018.